

Group 1: Reinforcement Learning in a High-Variance, Combinatorial World

Aidan Jared - ajar667

Daoyu Zhang - dzha977

Dominicus Nararya - dnar278

Jason Kong - jkon763

Thomas Clunie - tclu830

Yuanyuan Yu - yyu119

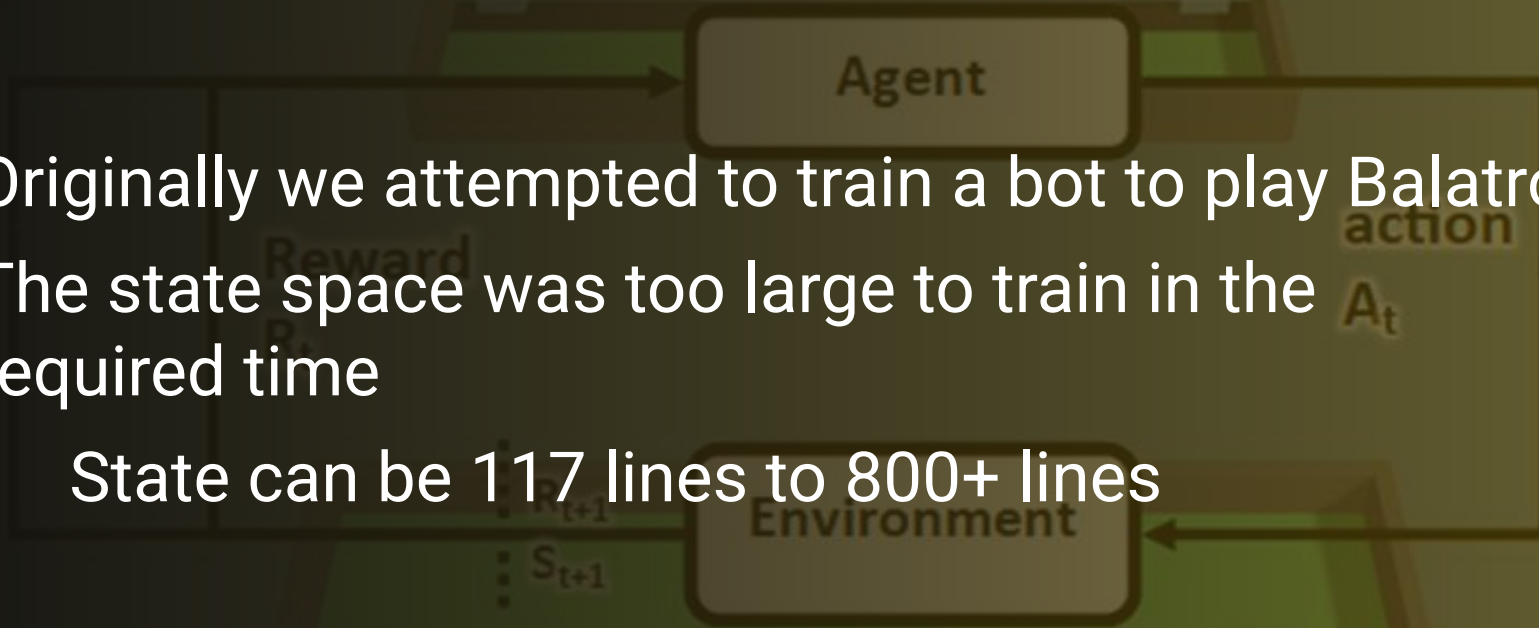


Major Updates

Originally we attempted to train a bot to play Balatro

The state space was too large to train in the required time

State can be 117 lines to 800+ lines



Major Updates



Atari Games can present equivalent challenges
for Hierarchical Reinforcement Learning

But they have standardized API and parallel
training environments

They are also baseline metrics in RL

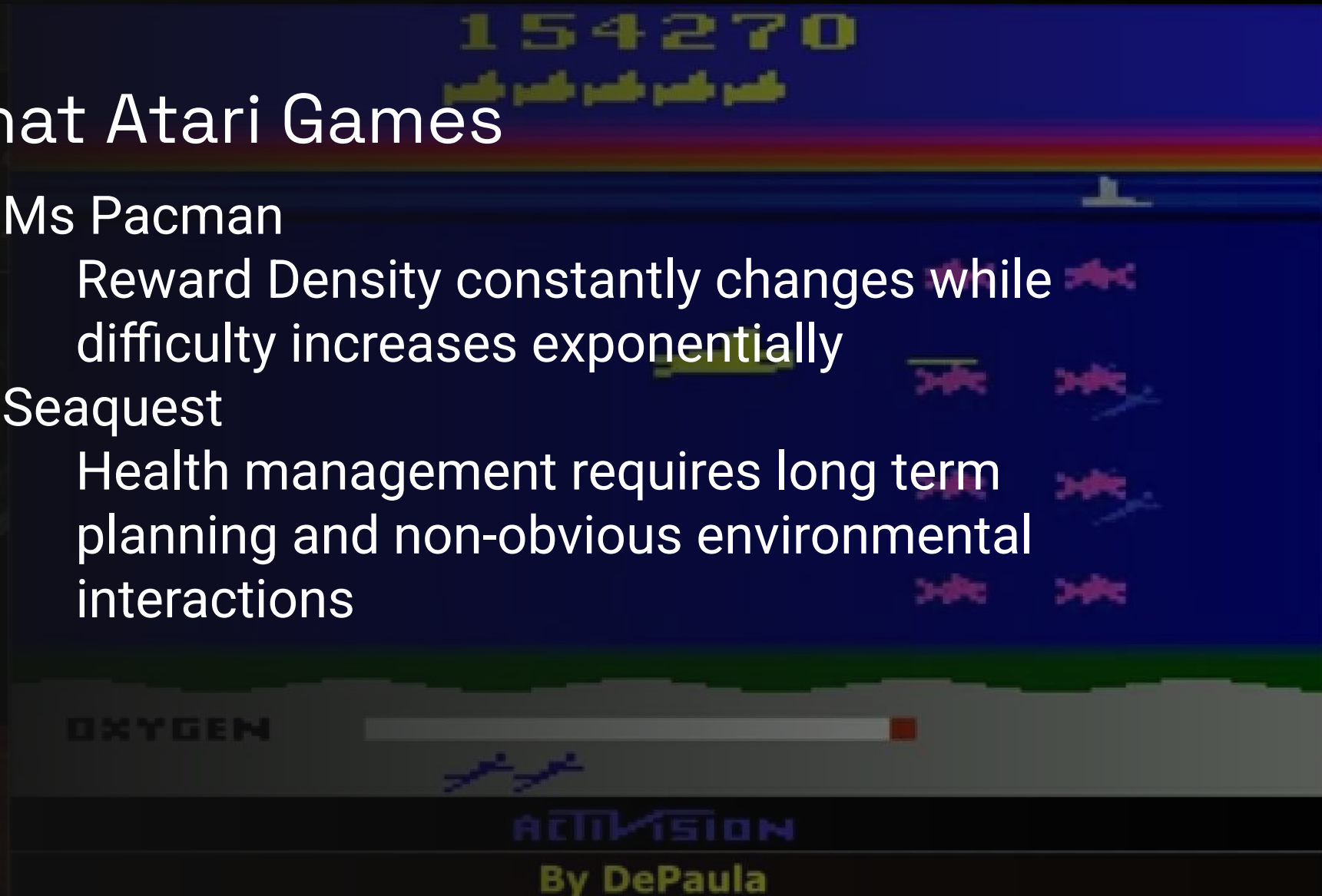
What Atari Games

Ms Pacman

Reward Density constantly changes while difficulty increases exponentially

Seaquest

Health management requires long term planning and non-obvious environmental interactions



What Atari Games

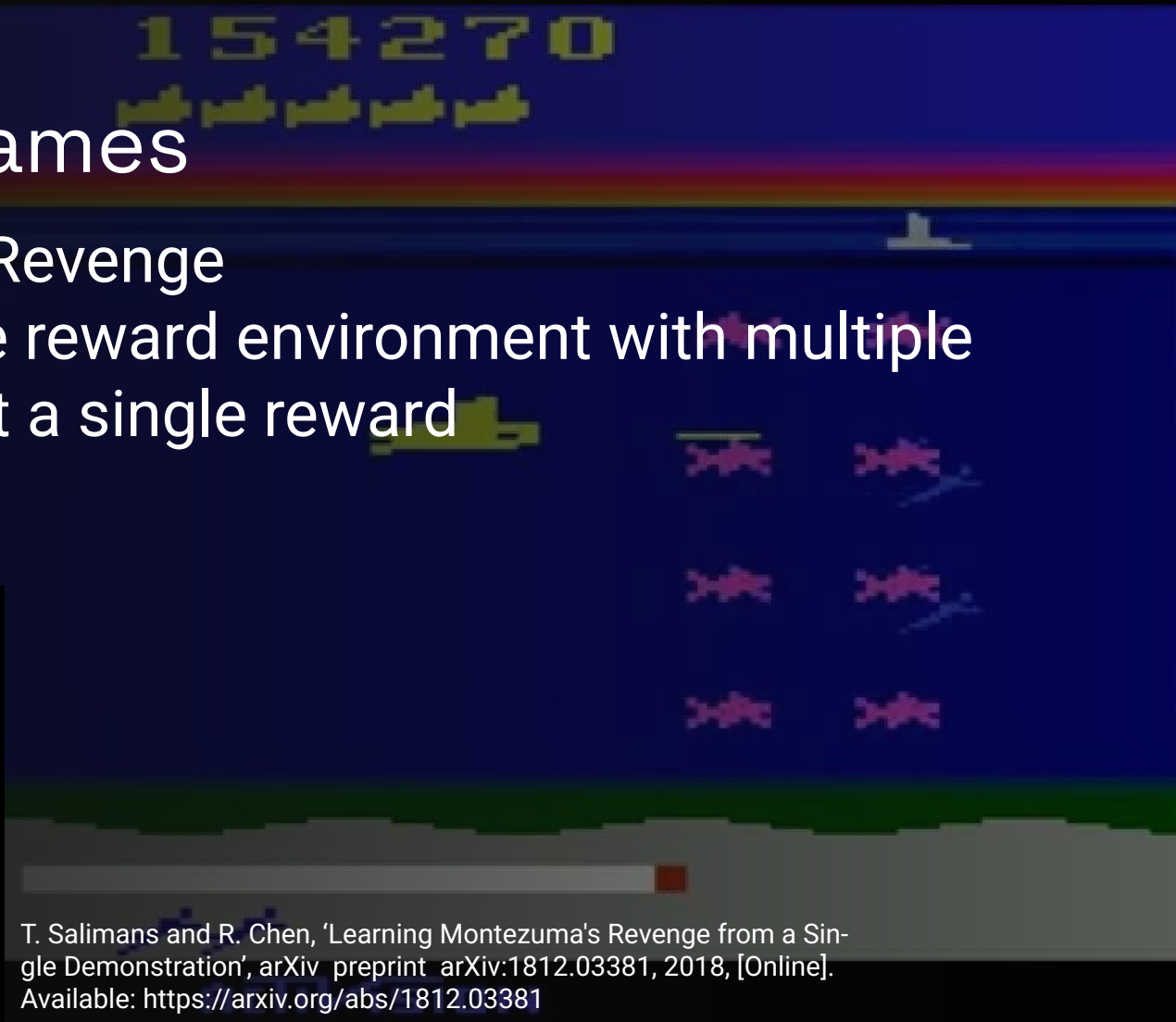
Montezuma's Revenge

Very sparse reward environment with multiple steps to get a single reward



T. Salimans and R. Chen, 'Learning Montezuma's Revenge from a Single Demonstration', arXiv preprint arXiv:1812.03381, 2018, [Online].
Available: <https://arxiv.org/abs/1812.03381>

By DePaula



Current Training Status

The training and testing environments have been designed

Using a 4050 GPU to train

Main limiting factor has been time and VRAM capacity

Need to design rewards for Seaquest and Montezuma's Revenge

Current Training Status

Hyperparameters:

Input Type: Image or RAM

Learning Rate: $1e-3$

Batch Size: 8 - 32 (depending on VRAM)

Current Training Status

Hyperparameters:

Reward Discount (how much to value past reward): .99 for worker, .999 for Manager

Time dilation for manager model (how far into the past to look): Currently using 10 for prototyping

Hidden and internal Sizes: 128, input is the same size

Experimental Setup

Unlike most ML problems, Online RL does not use train, test, and validation data

Focused more on designing good rewards and not learning bad behaviours

Experimental Setup

Standard evaluation metrics (RMSE, Accuracy, Cosine Similarity) cannot capture performance while mean and median scores are not statistically sound

The *rlia* package provides statistically sound evaluation metrics

Reward Design

Rewards dictate what the model does

Some Rewards can incentivise sub-optimal behaviour

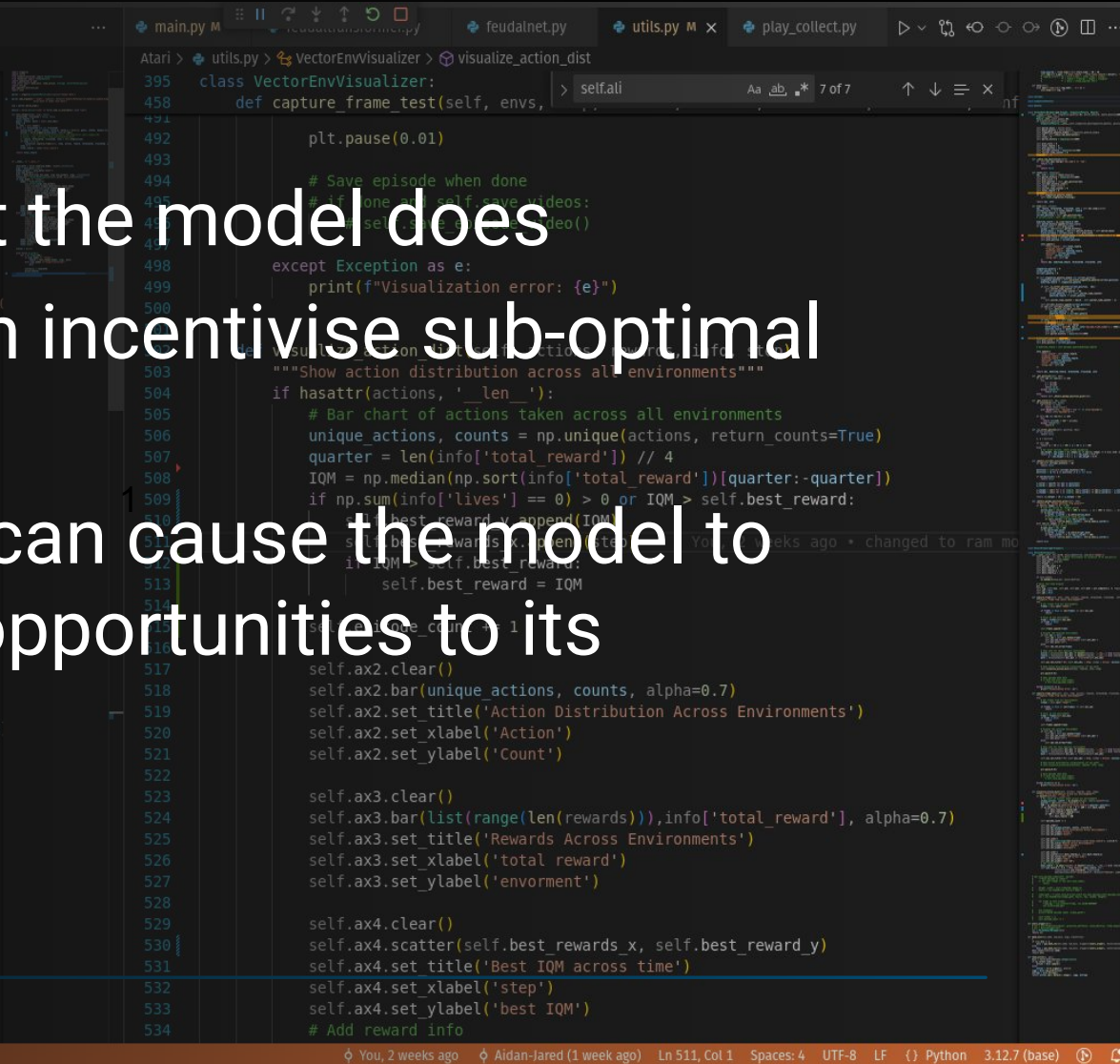
For example, they can cause the model to chase high score opportunities to its detriment

Reward Design

Rewards dictate what the model does

Some Rewards can incentivise sub-optimal behaviour

For example, they can cause the model to chase high score opportunities to its detriment



Learning the Wrong Signal

Models can learn the wrong signal from the environment or learn suboptimal interactions that work in training but fail in testing

```
def test_feudal(model, args, envs):
    terminated, truncated = False, False
    total_reward = 0
    goals, states, masks = model.init_obj()
    step = 0
    info = envs.reset()
    while not terminated and not truncated:
        action_dist, goals, states, value_m, value_w = model(x, goals, states, masks[-1])
        action = torch.argmax(action_dist, dim=-1).cpu()
        action = torch.multinomial(action_dist, num_samples=1).cpu().numpy()[0]
        action, logp, entropy = take_action(action_dist)
        x, reward, terminated, truncated, info = envs.step(action)
        if step % 4 == 1:
            visualizer.capture_frame(envs, step, action, reward, terminated, truncated, info)
            step += 1
            total_reward = info['total_reward']
    return total_reward

if __name__ == "__main__":
    save_data = torch.load(arg_model, weights_only=False)
    args = save_data['args']
    model_weights = save_data['model']
    args.num_workers = 1
    envs = make_envs(args.env_name, args.num_workers, args, train=False)
```

```
main.py M  feudalnet.py  utils.py M x  play_collect.py  Atari > utils.py > VectorEnvVisualizer > visualize_action_dist
class VectorEnvVisualizer:
    def capture_frame_test(self, envs,
        plt.pause(0.01)

    # Save episode when done
    if done and self.save_episodes:
        self.save_episode(self.episode_count)

    except Exception as e:
        print("Visualization error: {e}")

    def visualize_action_dist(self, actions, rewards, info, step):
        """Show action distribution across all environments"""
        if not self.save_episodes:
            return
        unique_actions, counts = np.unique(actions, return_counts=True)
        quarter = len(info['total_reward']) // 4
        IQM = np.median(np.sort(info['total_reward'])[quarter:-quarter])
        if np.sum(info['lives'] == 0) > 0 or IQM > self.best_reward:
            self.best_reward_y.append(IQM)
            self.best_rewards_x.append(step)
            if IQM > self.best_reward:
                self.best_reward = IQM

        self.episode_count += 1

        self.ax2.clear()
        self.ax2.bar(unique_actions, counts, alpha=0.7)
        self.ax2.set_title('Action Distribution Across Environments')
        self.ax2.set_xlabel('Action')
        self.ax2.set_ylabel('Count')

        self.ax3.clear()
        self.ax3.bar(list(range(len(rewards))), info['total_reward'], alpha=0.7)
        self.ax3.set_title('Rewards Across Environments')
        self.ax3.set_xlabel('total reward')
        self.ax3.set_ylabel('envorment')

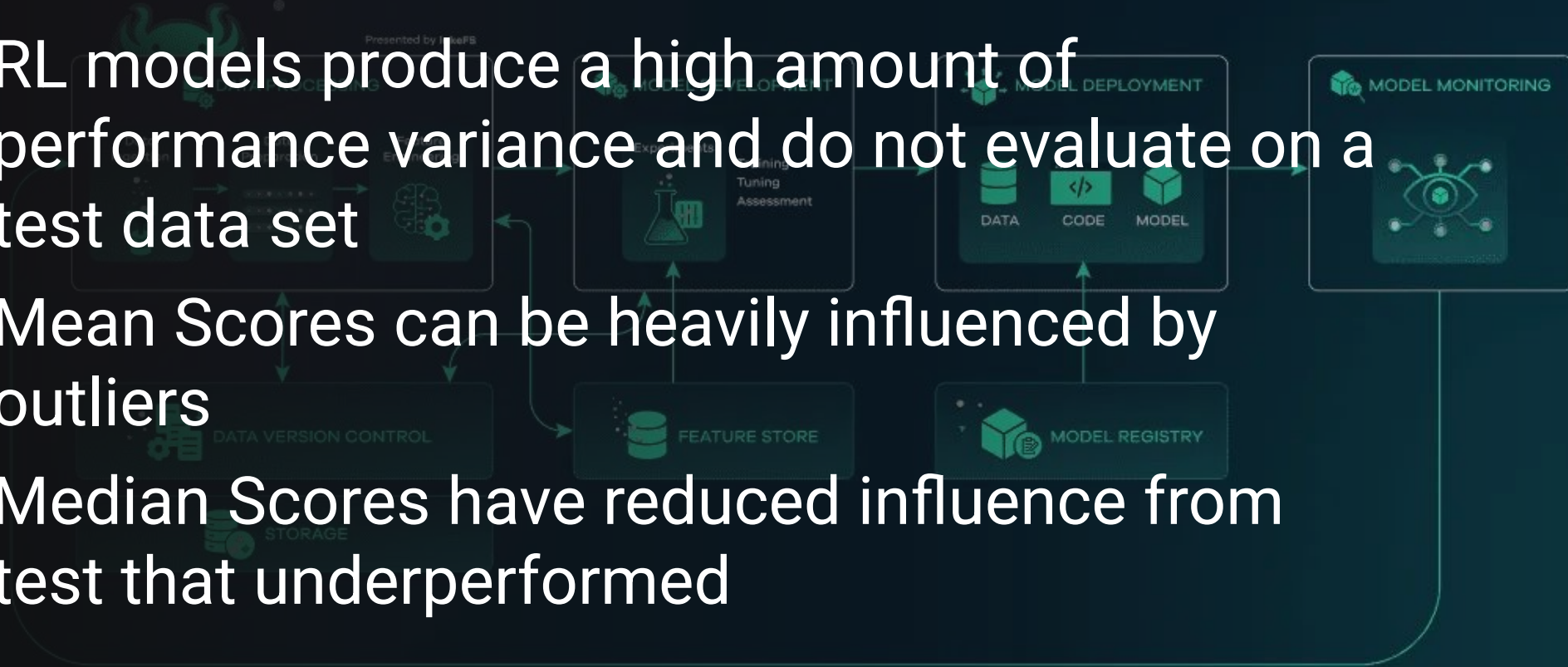
        self.ax4.clear()
        self.ax4.scatter(self.best_rewards_x, self.best_reward_y)
        self.ax4.set_title('Best IQM across time')
        self.ax4.set_xlabel('step')
        self.ax4.set_ylabel('best IQM')
        # Add reward info
```


Statistically Sound Metrics

RL models produce a high amount of performance variance and do not evaluate on a test data set

Mean Scores can be heavily influenced by outliers

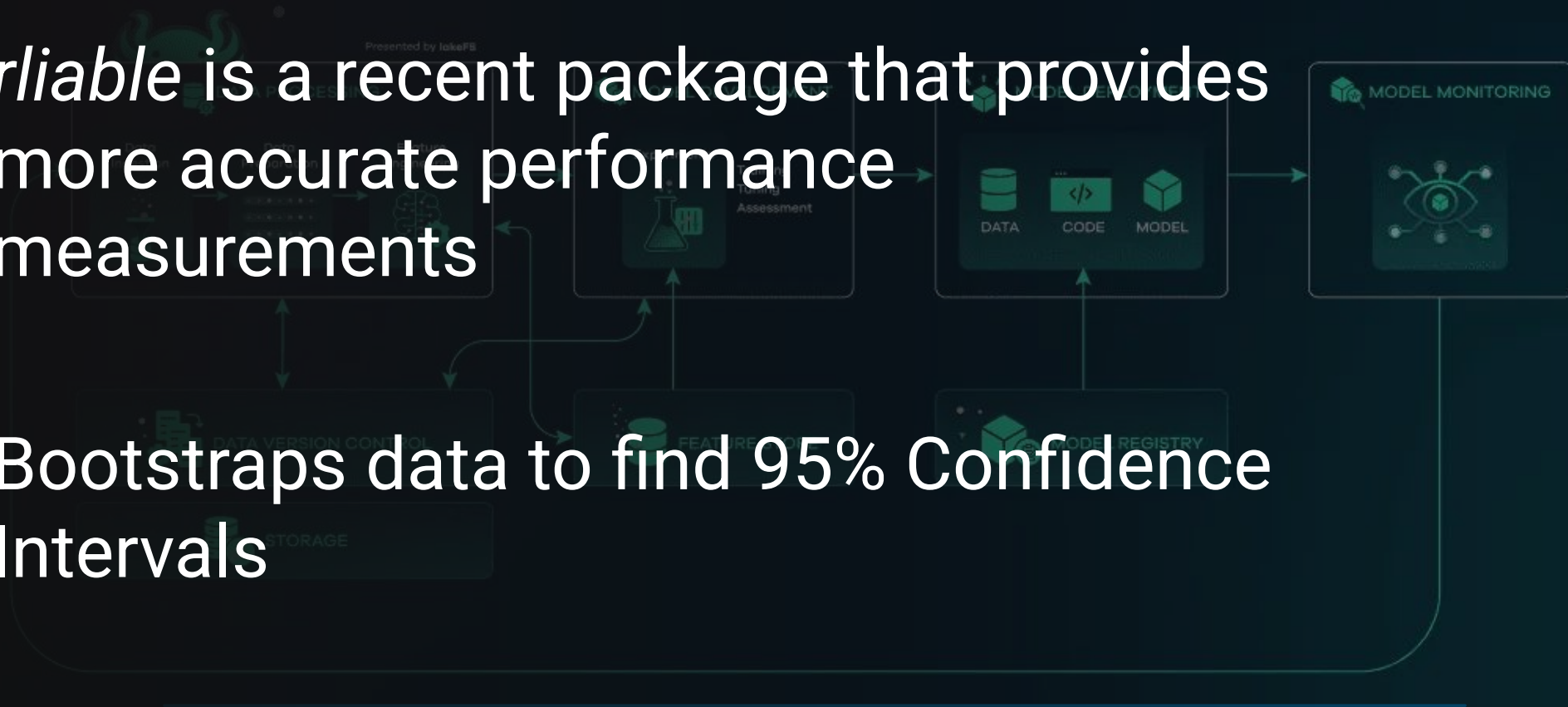
Median Scores have reduced influence from test that underperformed



Statistically Sound Metrics

reliable is a recent package that provides more accurate performance measurements

Bootstraps data to find 95% Confidence Intervals



Statistically Sound Metrics

Uses Inter Quartile Median to measure performance

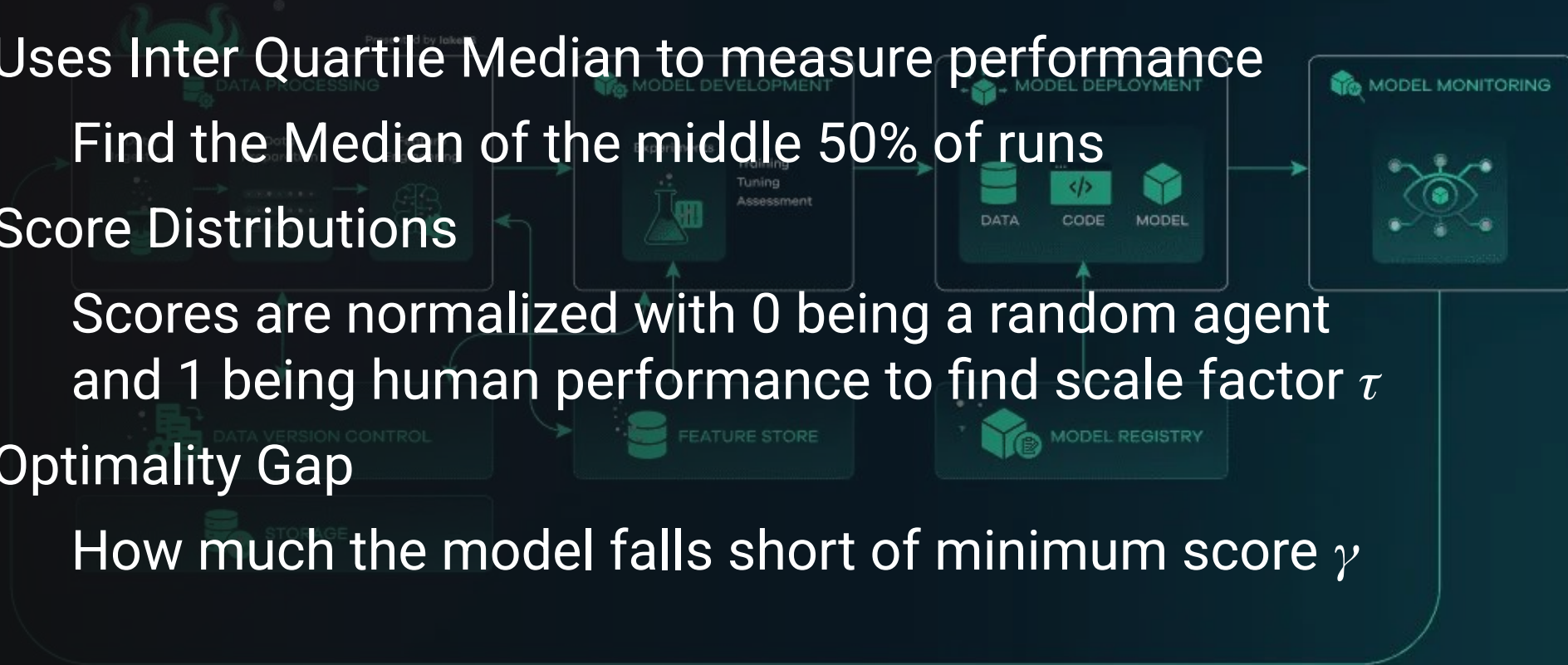
Find the Median of the middle 50% of runs

Score Distributions

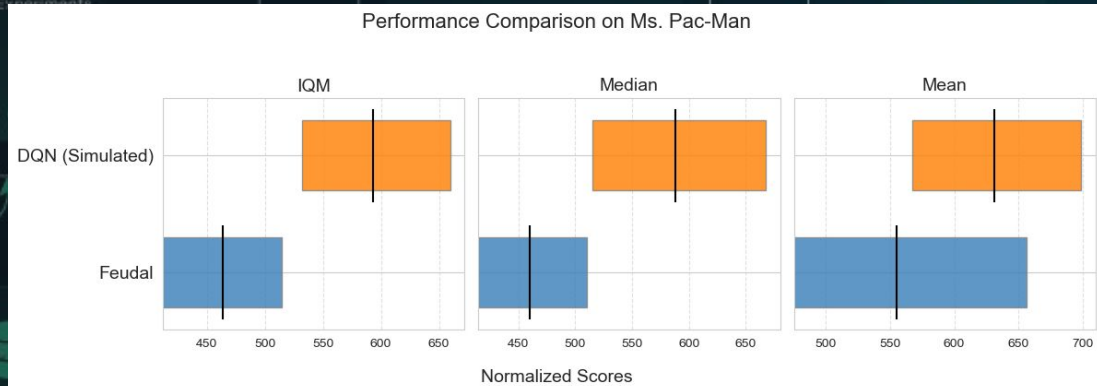
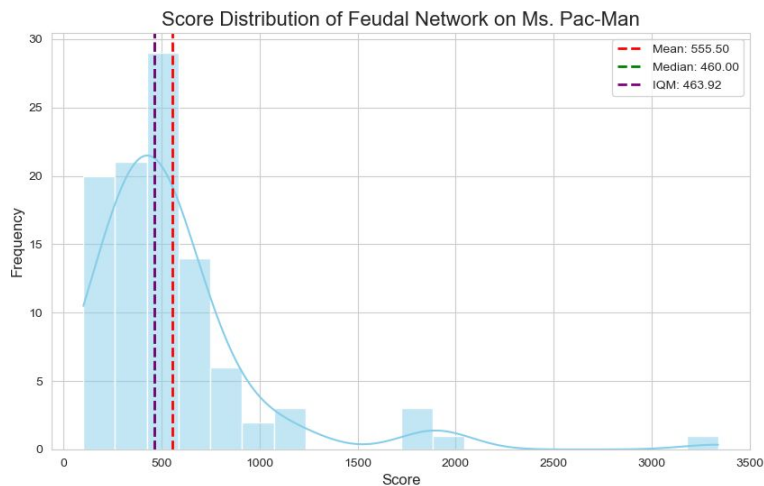
Scores are normalized with 0 being a random agent and 1 being human performance to find scale factor τ

Optimality Gap

How much the model falls short of minimum score γ



Example Model Evaluation



GitHub

https://github.com/Aidan-Jared/COMPSCI_760_Reinforcement/

```
Code Blame 345 lines (283 loc) · 12.4 KB
```

```
9 class FeudalNetwork(nn.Module):
10     def __init__(self, num_workers, input_dim, hidden_dim_manager, hidden_dim_worker, n_actions, time_horizon = 10, dilation = 10, device = 'cpu', mlp = False, args = None):
16         self.r = dilation
17         self.n_actions = n_actions
18         self.device = device
19         self.decay = args.decay
20
21         self.c = nn.Conv2d(input_dim, hidden_dim_manager, dilation=dilation)
22
23         self.perception = Perception(input_dim, self.d, mlp)
24         self.manager = Manager(self.c, self.d, self.r, args, device)
25         self.worker = Worker(self.b, self.c, self.d, self.k, n_actions, device, args)
26
27
28         self.hidden_m = self._init_hidden(args.num_workers, self.r * self.d, grad=True)
29         self.hidden_w = self._init_hidden(args.num_workers, self.k * n_actions, grad=True)
30
31         self.args = args
32         self.to(device)
33         self.apply(self._weight_init)
34
35     def _init_hidden(self, self, n_workers, h_dim, grad=False):
36         return(torch.zeros(n_workers, h_dim, requires_grad=grad).to(self.device), torch.zeros(n_workers, h_dim, requires_grad=grad).to(self.device))
37
38     def _weight_init(self, layer):
39         if type(layer) == nn.modules.conv.Conv2d or type(layer) == nn.Linear:
40             nn.init.orthogonal_(layer.weight.data)
41             if layer.bias is not None:
42                 nn.init.constant_(layer.bias.data, 0)
43
44     def forward(self, x, goals, states, mask, save=True):
45         x = self.preprocessor(x)
46         z = self.perception(x)
47         goal, hidden_m, state, value_m = self.manager(z, self.hidden_m, mask)
48
49         if len(goals) > (2 * self.c + 1):
50             goals.pop(0)
51             states.pop(0)
52
```

Questions?

